

Display Images & Menus

Contents

- **ImageView**
- **Display images**
- **Display menus**

ImageView

- **Key attributes:**
 - `android:id="@+id/theID"` unique ID for use in Java code
 - `android:src="@drawable/img"` image to put in the screen (must correspond to an image resource)
 - `android:scaleType`: options are used for scaling the bounds of an image to the bounds of image view

ImageView - scaleType

- **CENTER** : Displays the image centered in the view with no scaling.
- **CENTER_CROP** : Scales the image such that both the x and y dimensions are greater than or equal to the view, while maintaining the image aspect ratio; centers the image in the view
- **CENTER_INSIDE** : Scales the image to fit inside the view, while maintaining the image aspect ratio. If the image is already smaller than the view, then this is the same as center
- **FIT_CENTER** : Scales the image to fit inside the view, while maintaining the image aspect ratio. At least one axis will exactly match the view, and the result is centered inside the view
- **FIT_START** : Same as fitCenter but aligned to the top left of the view
- **FIT_END** : Same as fitCenter but aligned to the bottom right of the view
- **FIT_XY** : Scales the x and y dimensions to exactly match the view size. dDoes not maintain the image aspect ratio
- **MATRIX** : Scales the image using a supplied Matrix class. The matrix can be supplied using the setImageMatrix method. A Matrix class can be used to apply transformations such as rotations to an image

Display images from local file

- To change the visible image, in Java code:
 - get the `ImageView` using `findViewById`
 - call its **`setImageResource`** method and pass `R.drawable.filename`

Display images from URL - Picasso, Glide...

- Add Internet permission:

```
<uses-permission android:name="android.permission.INTERNET" />
```

- Setup

```
compile 'com.squareup.picasso:picasso:2.5.2'
```

- Use

```
if (url == null || url.equals("")) {  
    Picasso.with(context).load("error").placeholder(R.drawable.ic_loading)  
        .error(R.drawable.ic_loading)  
        .into(imageView);  
} else {  
    Picasso.with(context).load(url).placeholder(R.drawable.ic_loading)  
        .error(R.drawable.ic_loading)  
        .into(imageView);  
}
```

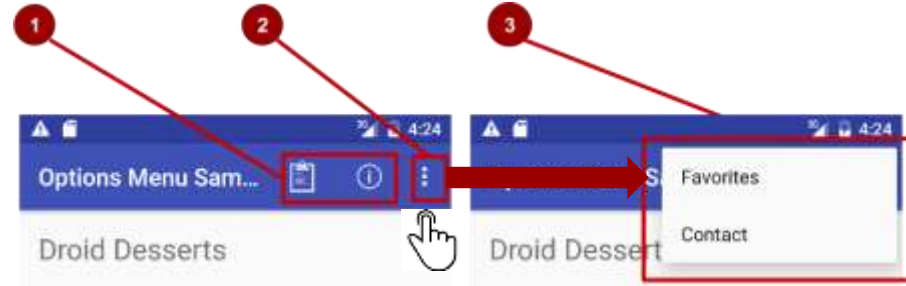
Display Menus

- **Types of menus:**
 - App bar with options menu
 - Context menu
 - Contextual action bar
 - Popup menu



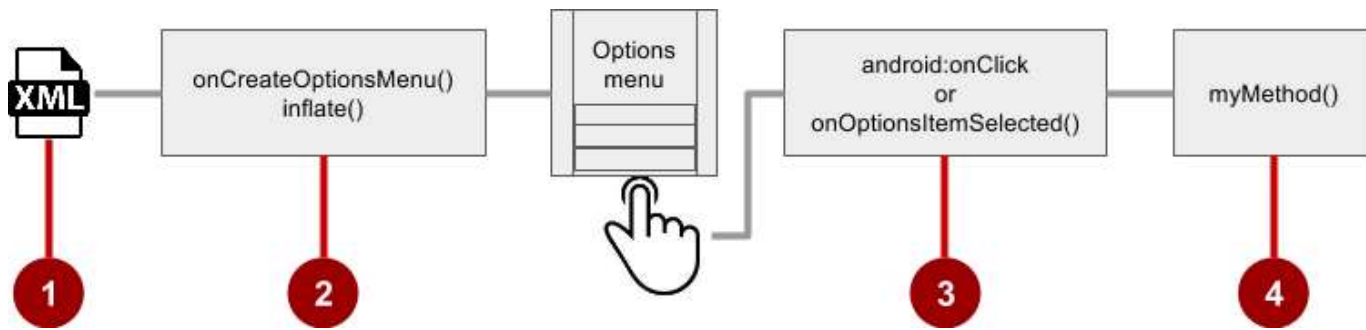
Options menu

- Action icons in the app bar for important items (1)
- Tap the three dots, the "action overflow button" to see the options menu (2)
- Appears in the right corner of the app bar (3)
- For navigating to other activities and editing app settings



Options menu - Steps to implement options menu

1. XML menu resource (menu_main.xml)
2. onCreateOptionsMenu() to inflate the menu
3. onClick attribute or onOptionsItemSelected()
4. Method to handle item click



Options menu - Create menu resource

1. Create menu resource directory
2. Create XML menu resource (menu_main.xml)
3. Add entry for each menu item (Settings and Favorites):

```
<item    android:id="@+id/option_settings"  
        android:title="Settings" />
```

```
<item    android:id="@+id/option_favorites"  
        android:title="Favorites" />
```

Options menu - Inflate options menu

- **Override onCreateOptionsMenu() in Activity**

```
@Override  
public boolean onCreateOptionsMenu(Menu menu) {  
    getMenuInflater().inflate(R.menu.menu_main, menu);  
    return true;  
}
```

Options menu - Add icons for menu items

1. Right-click drawable
2. Choose New > Image Asset
3. Choose Action Bar and Tab Items
4. Edit the icon name
5. Click clipart image, and click icon
6. Click Next, then Finish



Options menu - Add menu item attributes

`<item`

```
    android:id="@+id/action_favorites"  
    android:icon="@drawable/ic_favorite"  
    android:orderInCategory="30"  
    android:title="@string/action_favorites"  
    app:showAsAction="ifRoom" />
```

Options menu - Override onOptionsItemSelected()

@Override

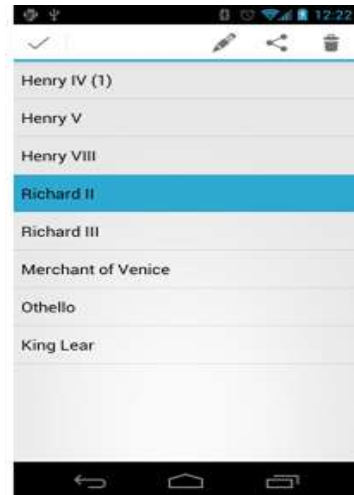
```
public boolean onOptionsItemSelected(MenuItem item) {  
    switch (item.getItemId()) {  
        case R.id.action_settings:  
            showSettings();  
            return true;  
        case R.id.action_favorites:  
            showFavorites();  
            return true;  
        default:  
            return super.onOptionsItemSelected(item);  
    }  
}
```

Contextual menus

- Allows users to perform action on selected View
- Can be deployed on any View
- Most often used for items in RecyclerView, GridView, or other View collection

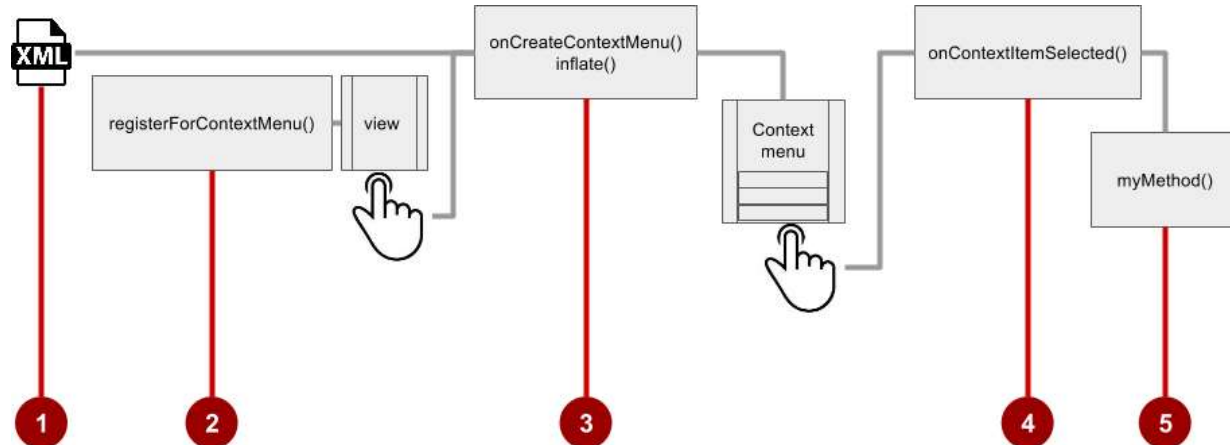
Types of contextual menus

- **Floating context menu—long-press on a View**
 - User can modify View or use it in some fashion
 - User performs action on one View at a time
- **Contextual action mode—temporary action bar in place of or underneath app bar**
 - Action items affect the selected View element(s)
 - User can perform action on multiple View elements at once



Context Menu implementation Steps

1. Create XML menu resource file and assign appearance and position attributes
2. Register View using `registerForContextMenu()`
3. Implement `onCreateContextMenu()` in Activity to inflate menu
4. Implement `onContextItemSelected()` to handle menu item clicks
5. Create method to perform action for each context menu item



1. Create menu resource

- Create XML menu resource (menu_context.xml)

```
<item
    android:id="@+id/context_edit"
    android:title="Edit"
    android:orderInCategory="10"/>
```

```
<item
    android:id="@+id/context_share"
    android:title="Share"
    android:orderInCategory="20"/>
```

2. Register a view to a context menu

- In onCreate() of the Activity, Register [View.OnCreateContextMenuListener](#) to View:

```
TextView article_text = findViewById(R.id.article);  
registerForContextMenu(article_text);
```

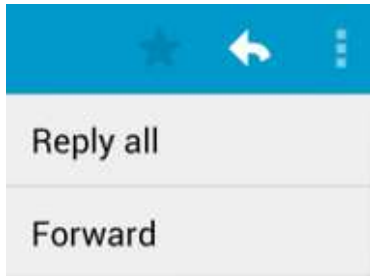
3. Implement onCreateContextMenu()

- Specify which context menu

@Override

```
public void onCreateContextMenu(ContextMenu menu, View v,  
                                ContextMenu.ContextMenuInfo menuInfo) {  
    super.onCreateContextMenu(menu, v, menuInfo);  
    MenuInflater inflater = getMenuInflater();  
    inflater.inflate(R.menu.menu_context, menu);  
}
```

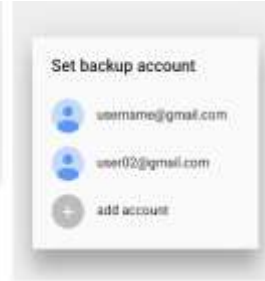
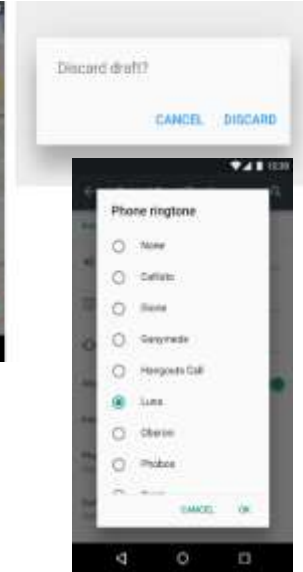
More things



Popup Menu



Picker



Dialog

Classwork

- **Create menu for Product List screen (slide 9) with 2 menu items:**
 - **Search**
 - **Cart**
- **Create Product Details screen:**
 - **Show product information, image**

References

- <https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/>